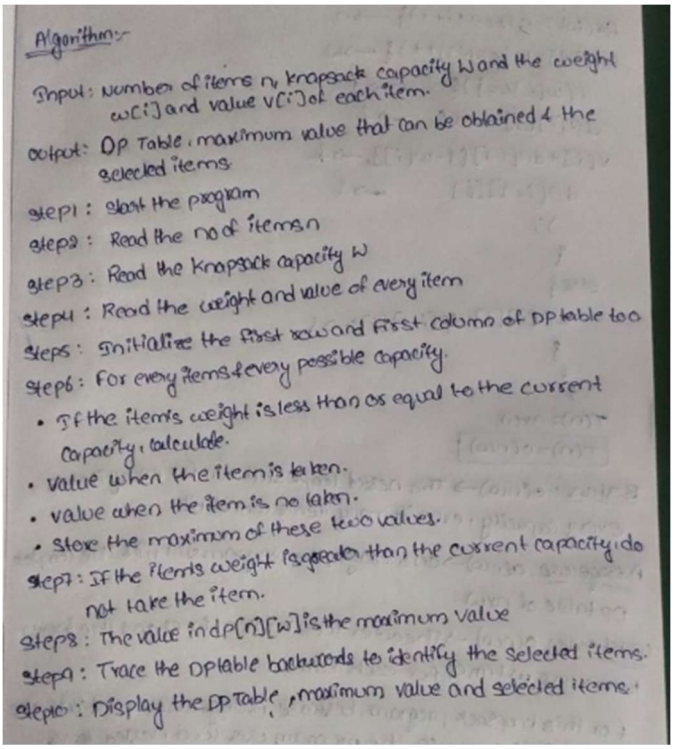
Practical-5

Aim: Implementation of a knapsack problem using dynamic programming.

Algorithm:



Algorithm:-

Input: Number of items n, knapsack capacity W and the weight w[i] and value v[i] of each item.

output: DP Table, maximum value that can be obtained & the selected items

step1 : Start the program

step2 : Read the no of items n

step3 : Read the knapsack capacity W

step4 : Read the weight and value of every item

step5 : Initialize the first row and first column of DP table to 0

step6 : For every items & every possible capacity

- If the items weight is less than or equal to the current capacity, calculate.
- value when the item is taken.
- value when the item is no taken.
- Store the maximum of these two values.

step7 : If the item's weight is greater than the current capacity, do not take the item.

step8 : The value in dp[n][w] is the maximum value

step9 : Trace the DP table backwards to identify the selected items.

step10 : Display the DP Table, maximum value and selected items.

Program:

```
#include <iostream>

#include <vector>

using namespace std;


int knapsack(int W, vector<int> wt, vector<int> val, int n) {

    vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));
```

```
    for (int i = 1; i <= n; i++) {
        for (int w = 1; w <= W; w++) {
            if (wt[i - 1] <= w) {
                dp[i][w] = max(val[i - 1] + dp[i - 1][w - wt[i - 1]],
                               dp[i - 1][w]);
            } else {
                dp[i][w] = dp[i - 1][w];
            }
        }
    }

    return dp[n][W];
}

int main() {
    int n, W;

    cout << "Enter number of items: ";
    cin >> n;

    vector<int> wt(n), val(n);

    cout << "Enter weights of items:\n";
    for (int i = 0; i < n; i++)
        cin >> wt[i];
```

```
cout << "Enter values of items:\n";

    cout << "Enter knapsack capacity: ";
    cin >> W;

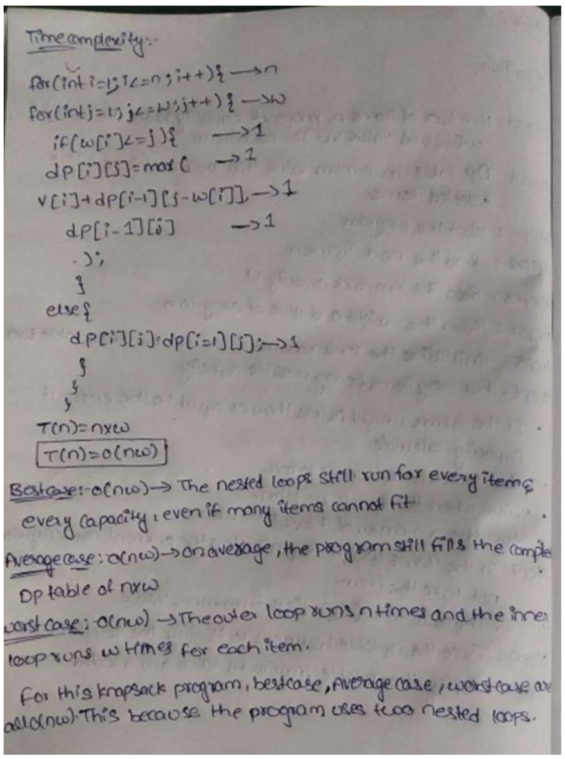

    cout << "Maximum value = " << knapsack(W, wt, val, n) << endl;


    return 0;
}
```

Output:

```
Enter number of items: 4
Enter weights of items:
5
7
8
9
Enter values of items:
5
6
7
8
Enter knapsack capacity: 4
Maximum value = 0
```

Time complexity:



Conclusion:

The 0/1 Knapsack problem was successfully implemented using the Dynamic Programming technique. The program calculates the maximum value that can be obtained within the given knapsack capacity and identifies the selected items. The time and space complexity of the program are O(nW).

Name:B.Vennela
Enrollment No:92460118232